



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	ARIANNE BEVERLY MARQUEZ
Section:	AN41
Professor:	MS. CRISOLA TAN
Date:	10/01/26

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
(Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
 - Classes and objects

- If-Else statements
 - Loops (for loop)
 - Functions / methods
2. Understand the environment:
- Two rooms: A and B
 - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output

```
C:\Users\hazel\AppData\Local... × + ▾  
Is room A Dirty or Clean? dirty  
Is room B Dirty or Clean? clean  
  
How many steps should the agent run? 5  
  
Step 1: Agent is in room A  
Room A has been cleaned.  
Environment state: {'A': 'Clean', 'B': 'Clean'}  
  
Step 2: Agent is in room A  
Moving to room B  
Environment state: {'A': 'Clean', 'B': 'Clean'}  
  
Step 3: Agent is in room B  
Moving to room A  
Environment state: {'A': 'Clean', 'B': 'Clean'}  
  
Step 4: Agent is in room A  
Moving to room B  
Environment state: {'A': 'Clean', 'B': 'Clean'}  
  
Step 5: Agent is in room B  
Moving to room A  
Environment state: {'A': 'Clean', 'B': 'Clean'}  
Press any key to continue . . . |
```

Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
 - Rule 1: If the current room is dirty, clean it.
 - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
 - The agent's current location
 - The action taken (cleaned/moved)
 - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)

```
C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Dirty] [C-Dirty]
Moving to room B
[A-Clean] [B-Agent] [C-Dirty]

Step 2:
[A-Clean] [B-Agent] [C-Dirty]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Dirty]

Step 3:
[A-Clean] [B-Agent] [C-Dirty]
Moving to room C
[A-Clean] [B-Clean] [C-Agent]

Step 4:
[A-Clean] [B-Clean] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]

Step 5:
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
Press any key to continue . . . |
```

Your Code - Laboratory Tasks

❖ Task 1

➤ Code

```
print('\nProgrammed by: Arianne Beverly V. Marquez\n')
```

```

class VacuumEnvironment:
    def __init__(self):
        # Initialize two rooms using a Python
        dictionary.
        # By default, both are clean, but user input
        will overwrite this.
        self.rooms = {'A': 'Clean', 'B': 'Clean'}

        # The agent defaults to starting in room A.
        self.current_room = 'A'

    def set_initial_states(self):
        # Allow the user to set the initial state of
        each room (dirty or clean).
        # .capitalize() ensures the formatting matches
        the dictionary output (e.g., 'Dirty', 'Clean')
        state_a = input("\nIs room A Dirty or Clean?
\n").strip().capitalize()
        state_b = input("\nIs room B Dirty or Clean?
\n").strip().capitalize()
        self.rooms['A'] = state_a
        self.rooms['B'] = state_b

    def is_dirty(self, room):
        # Function to check if a room is dirty.
        return self.rooms[room] == 'Dirty'

    def clean_room(self, room):
        # Function to clean a room.
        self.rooms[room] = 'Clean'
        print(f"Room {room} has been cleaned.")

    def run_agent(self, steps):
        # Loop (for loop) to simulate the agent
        running for the specified number of steps.
        for step in range(1, steps + 1):
            print(f"\nStep {step}: Agent is in room
            {self.current_room}")

            # If-Else statements to determine the
            agent's action
            if self.is_dirty(self.current_room):
                self.clean_room(self.current_room)
            else:

```

```

        # If the room is already clean, move
to the other room
        next_room = 'B' if self.current_room
== 'A' else 'A'
        print(f"Moving to room {next_room}")
        self.current_room = next_room

        # Display the exact environment state
after the action is taken
        print(f"Environment state: {self.rooms}")

# Main program execution
if __name__ == "__main__":
    while True:
        # Create a fresh environment for each run so
the agent resets to Room A
        env = VacuumEnvironment()

        # Prompt user for initial room states
        env.set_initial_states()
        print()

        # Ask the user for the number of steps
        try:
            steps = int(input("How many steps should
the agent run? "))
        except ValueError:
            print("Invalid input. Defaulting to 5
steps.")
            steps = 5

        # Run the agent through the environment
        env.run_agent(steps)

        print()
        input("Press any key to continue . . . ")

        # Ask the user if they want to run it again
        print()
        retry = input("Would you like to run the
simulation again? (Type 'yes' to restart, or any other
key to exit): ").strip().lower()

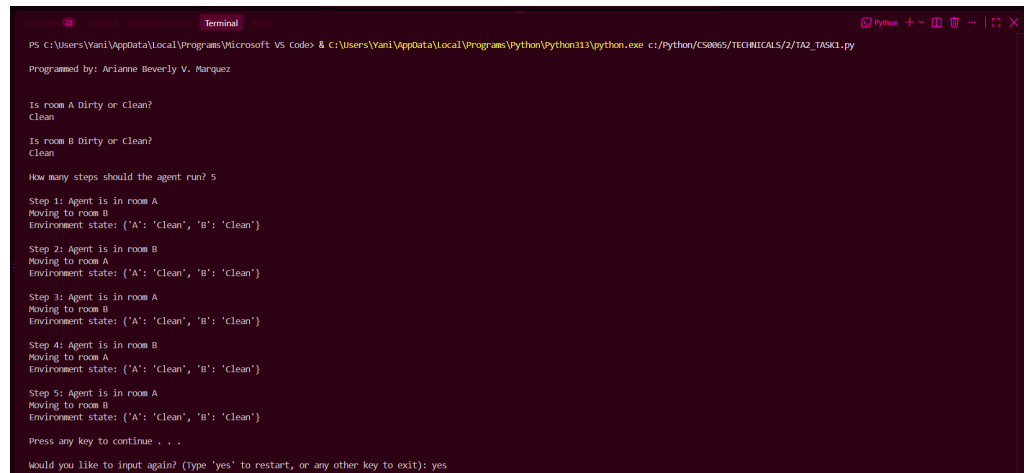
        if retry not in ['yes', 'y']:

```

```
print("Exiting program. Goodbye!")
break
```

```
# Print a visual separator for the next run
print("\n" + "="*41 + "\n")
```

➤ Output



```
PS C:\Users\Yani\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Yani\AppData\Local\Programs\Python\Python313\python.exe c:/Python/CS0065/TECHNICALS/2/TA2_TASK1.py
Programmed by: Arianne Beverly V. Marquez

Is room A Dirty or Clean?
Clean

Is room B Dirty or Clean?
Clean

How many steps should the agent run? 5

Step 1: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Press any key to continue . . .
Would you like to input again? (Type 'yes' to restart, or any other key to exit): yes
```

- Both rooms clean

Is room A Dirty or Clean?

Clean

Is room B Dirty or Clean?

Clean

How many steps should the agent run? 5

Step 1: Agent is in room A

Moving to room B

Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room B

Moving to room A

Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room A

Moving to room B

Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room B

Moving to room A

Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room A

Moving to room B

Environment state: {'A': 'Clean', 'B': 'Clean'}

Press any key to continue . . .

- One room dirty, one room clean

```
Is room A Dirty or Clean?  
Dirty
```

```
Is room B Dirty or Clean?  
Clean
```

```
How many steps should the agent run? 5
```

```
Step 1: Agent is in room A  
Room A has been cleaned.  
Environment state: {'A': 'Clean', 'B': 'Clean'}
```

```
Step 2: Agent is in room A  
Moving to room B  
Environment state: {'A': 'Clean', 'B': 'Clean'}
```

```
Step 3: Agent is in room B  
Moving to room A  
Environment state: {'A': 'Clean', 'B': 'Clean'}
```

```
Step 4: Agent is in room A  
Moving to room B  
Environment state: {'A': 'Clean', 'B': 'Clean'}
```

```
Step 5: Agent is in room B  
Moving to room A  
Environment state: {'A': 'Clean', 'B': 'Clean'}
```

```
Press any key to continue . . .
```

■ Both rooms dirty

```
Is room A Dirty or Clean?
Dirty

Is room B Dirty or Clean?
Dirty

How many steps should the agent run? 5

Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Dirty'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Dirty'}

Step 3: Agent is in room B
Room B has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Press any key to continue . . .

Would you like to run the simulation again? (Type 'yes' to restart, or any other key to exit):
Exiting program. Goodbye!
```

❖ Task 2

➤ Code

```
# Task 2
```

```
print('\nProgrammed by: Arianne Beverly V. Marquez\n')
```

```
class VacuumEnvironment:
    def __init__(self):
        # Initialize two rooms using a Python
        dictionary.
        self.rooms = {'A': 'Clean', 'B': 'Clean'}
```

```
    def set_initial_states(self):
        # Allow the user to set the initial state of
        each room (dirty or clean).
```

```

        state_a = input("\nIs room A Dirty or Clean?
\n").strip().capitalize()
        state_b = input("\nIs room B Dirty or Clean?
\n").strip().capitalize()
        self.rooms['A'] = state_a
        self.rooms['B'] = state_b

    def is_dirty(self, room):
        # Function to check if a room is dirty.
        return self.rooms[room] == 'Dirty'

    def clean_room(self, room):
        # Function to clean a room.
        self.rooms[room] = 'Clean'
        print(f"Room {room} has been cleaned.")

    def run_agent(self, steps):
        # We now create the Agent from Task 2 here to
run the loop
        agent = VacuumAgent(self)

        # Loop (for loop) to simulate the agent
running
        for step in range(1, steps + 1):
            print(f"\nStep {step}: Agent is in room
{agent.current_room}")

            # The agent perceives its environment and
acts based on its rules
            agent.perceive_and_act()

            # Display the exact environment state
after the action is taken
            print(f"Environment state: {self.rooms}")

# ADDED FOR TASK 2: The Agent Class

class VacuumAgent:
    def __init__(self, environment):
        self.env = environment
        self.current_room = 'A'

    def move(self):

```

```

        # Implement a move method to switch rooms.
        self.current_room = 'B' if self.current_room
== 'A' else 'A'
        print(f"Moving to room {self.current_room}")

    def perceive_and_act(self):
        # Implement the perceive-and-act method.
        # Rule 1: If the current room is dirty, clean
it.
        if self.env.is_dirty(self.current_room):
            self.env.clean_room(self.current_room)
        # Rule 2: If the current room is clean, move
to the other room.
        else:
            self.move()

# Main program execution (Kept exactly as your Task
1!)
if __name__ == "__main__":
    while True:
        # Create a fresh environment for each run
        env = VacuumEnvironment()

        # Prompt user for initial room states
        env.set_initial_states()
        print()

        # Ask the user for the number of steps
        try:
            steps = int(input("How many steps should
the agent run? "))
        except ValueError:
            print("Invalid input. Defaulting to 5
steps.")
            steps = 5

        # Run the agent through the environment
        env.run_agent(steps)

        print()
        input("Press any key to continue . . . ")

        # Ask the user if they want to run it again

```

```

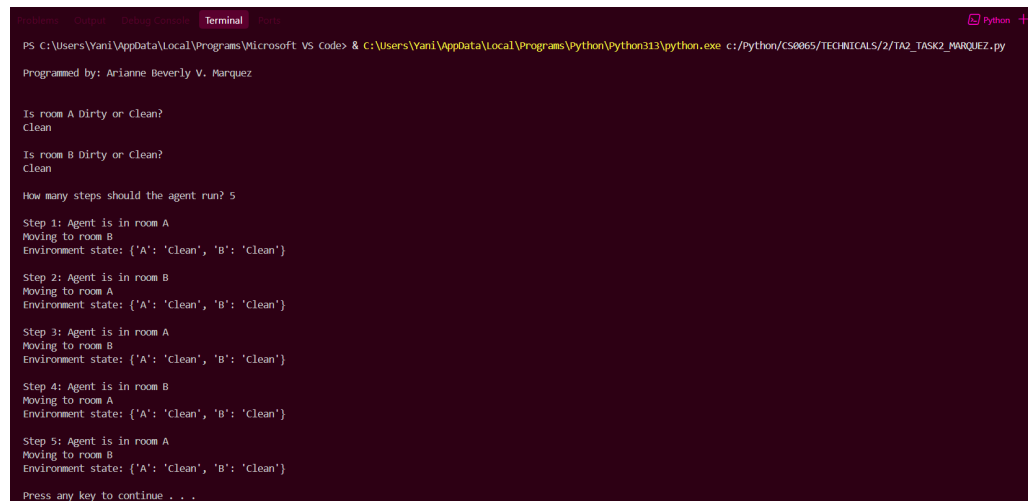
        print()
        retry = input("Would you like to run the
simulation again? (Type 'yes' to restart, or any other
key to exit): ").strip().lower()

        if retry not in ['yes', 'y']:
            print("Exiting program. Goodbye!")
            break

        # Print a visual separator for the next run
        print("\n" + "="*41 + "\n")

```

➤ Output



```

PS C:\Users\Yani\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Yani\AppData\Local\Programs\Python\Python313\python.exe c:/Python/CS0065/TECHNICALS/2/TA2_TASK2_MARQUEZ.py
Programmed by: Arlaine Beverly V. Marquez

Is room A Dirty or Clean?
Clean

Is room B Dirty or Clean?
Clean

How many steps should the agent run? 5

Step 1: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Press any key to continue . . .

```

- Both rooms clean

Programmed by: Arianne Beverly V. Marquez

Is room A Dirty or Clean?
Clean

Is room B Dirty or Clean?
Clean

How many steps should the agent run? 5

Step 1: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Press any key to continue . . .

- One room is clean, the other is dirty

```
Is room A Dirty or Clean?  
Dirty
```

```
Is room B Dirty or Clean?  
Clean
```

```
How many steps should the agent run? 5
```

```
Step 1: Agent is in room A  
Room A has been cleaned.  
Environment state: {'A': 'Clean', 'B': 'Clean'}
```

```
Step 2: Agent is in room A  
Moving to room B  
Environment state: {'A': 'Clean', 'B': 'Clean'}
```

```
Step 3: Agent is in room B  
Moving to room A  
Environment state: {'A': 'Clean', 'B': 'Clean'}
```

```
Step 4: Agent is in room A  
Moving to room B  
Environment state: {'A': 'Clean', 'B': 'Clean'}
```

```
Step 5: Agent is in room B  
Moving to room A  
Environment state: {'A': 'Clean', 'B': 'Clean'}
```

```
Press any key to continue . . .
```

- Both rooms are dirty

```

Is room A Dirty or Clean?
Dirty

Is room B Dirty or Clean?
Dirty

How many steps should the agent run? 5

Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Dirty'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Dirty'}

Step 3: Agent is in room B
Room B has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Press any key to continue . . .

Would you like to run the simulation again? (Type 'yes' to restart, or any other key to exit):
Exiting program. Goodbye!

```

- ❖ Task 3

- Yellow

- Code

```
# Task 3
```

```
print('\nProgrammed by: Arianne Beverly V. Marquez\n')
```

```
# TASK 1: THE ENVIRONMENT
```

```

class VacuumEnvironment:
    def __init__(self):
        # Initialize two rooms using a Python
        dictionary.
        # By default, both are clean, but user input
        will overwrite this.
        self.rooms = {'A': 'Clean', 'B': 'Clean'}

```

```

    def set_initial_states(self):
        # Allow the user to set the initial state of
        # each room (dirty or clean).
        state_a = input("\nIs room A Dirty or Clean?
\n").strip().capitalize()
        state_b = input("\nIs room B Dirty or Clean?
\n").strip().capitalize()
        self.rooms['A'] = state_a
        self.rooms['B'] = state_b

```

```

    def is_dirty(self, room):
        # Function to check if a room is dirty.
        return self.rooms[room] == 'Dirty'

```

```

    def clean_room(self, room):
        # Function to clean a room.
        self.rooms[room] = 'Clean'

```

TASK 2: THE RULE-BASED AGENT

```

class VacuumAgent:
    def __init__(self, environment):
        # Pass the environment to the agent so it can
        # interact with the rooms
        self.env = environment
        # The agent defaults to starting in room A.
        self.current_room = 'A'

```

```

    def move(self):
        # Implement a move method to switch rooms.
        self.current_room = 'B' if self.current_room
        == 'A' else 'A'
        # Print the action taken for Task 3 output
        print(f"Action taken: Moved to room
        {self.current_room}")

```

```

    def perceive_and_act(self):
        # Implement the perceive-and-act method.
        # Rule 1: If the current room is dirty, clean
        # it.
        if self.env.is_dirty(self.current_room):
            self.env.clean_room(self.current_room)

```

```

        # Print the action taken for Task 3 output
        print("Action taken: Cleaned the room.")
        # Rule 2: If the current room is clean, move
to the other room.
        else:
            self.move()

```

```

# TASK 3: RUN THE SIMULATION

```

```

if __name__ == "__main__":
    while True:
        # Create a fresh environment and agent for
each run
        env = VacuumEnvironment()
        agent = VacuumAgent(env)

```

```

        # Prompt user for initial room states
        env.set_initial_states()
        print()

```

```

        # Ask the user how many steps the simulation
should run.
        try:
            steps = int(input("How many steps should
the agent run? "))
        except ValueError:
            print("Invalid input. Defaulting to 5
steps.")
            steps = 5

```

```

        # Run the agent in a loop for the specified
steps.
        for step in range(1, steps + 1):
            print(f"\n--- Step {step} ---")

```

```

            # 1. Print the agent's current location
            print(f"Agent's current location: Room
{agent.current_room}")

```

```

            # 2. Print the action taken (handled
inside perceive_and_act and move methods)
            agent.perceive_and_act()

```

```

# 3. Print the current state of the
environment
print(f"Current state of the environment:
{env.rooms}")

print()
input("Press any key to continue . . . ")

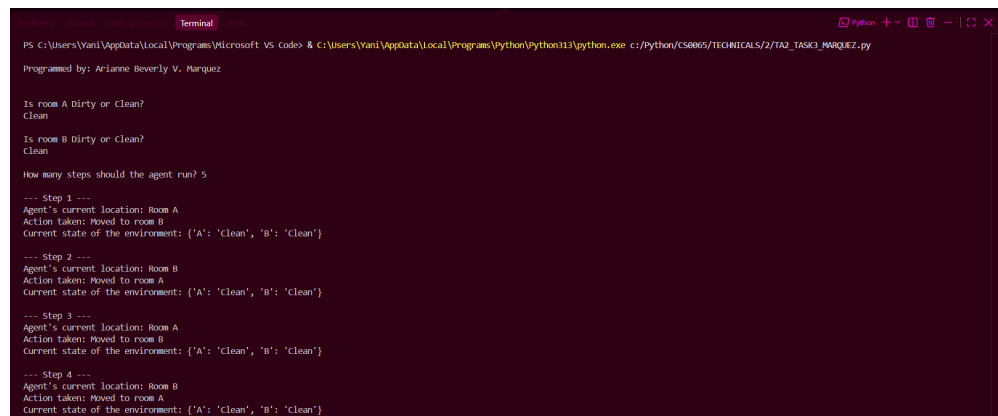
# Ask the user if they want to run it again
print()
retry = input("Would you like to run the
simulation again? (Type 'yes' to restart, or any other
key to exit): ").strip().lower()

if retry not in ['yes', 'y']:
    print("Exiting program. Goodbye!")
    break

# Print a visual separator for the next run
print("\n" + "="*41 + "\n")

```

➤ Output



```

Terminal
PS C:\Users\Vani\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Vani\AppData\Local\Programs\Python\Python113\python.exe c:/Python/CS0065/TECHNICALS/2/T2A_TASK3_MARQUEZ.py
Programmed by: Arianne Beverly V. Marquez

Is room A Dirty or Clean?
Clean
Is room B Dirty or Clean?
Clean
How many steps should the agent run? 5

--- Step 1 ---
Agent's current location: Room A
Action taken: Moved to room B
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

--- Step 2 ---
Agent's current location: Room B
Action taken: Moved to room A
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

--- Step 3 ---
Agent's current location: Room A
Action taken: Moved to room B
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

--- Step 4 ---
Agent's current location: Room B
Action taken: Moved to room A
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

```

- Both rooms clean

```
Programmed by: Arianne Beverly V. Marquez

Is room A Dirty or Clean?
Clean

Is room B Dirty or Clean?
Clean

How many steps should the agent run? 5

--- Step 1 ---
Agent's current location: Room A
Action taken: Moved to room B
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

--- Step 2 ---
Agent's current location: Room B
Action taken: Moved to room A
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

--- Step 3 ---
Agent's current location: Room A
Action taken: Moved to room B
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

--- Step 4 ---
Agent's current location: Room B
Action taken: Moved to room A
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

--- Step 5 ---
Agent's current location: Room A
Action taken: Moved to room B
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

Press any key to continue . . .
```

- One room dirty, the other is clean

```
Is room A Dirty or Clean?
Dirty

Is room B Dirty or Clean?
Clean

How many steps should the agent run? 5

--- Step 1 ---
Agent's current location: Room A
Action taken: Cleaned the room.
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

--- Step 2 ---
Agent's current location: Room A
Action taken: Moved to room B
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

--- Step 3 ---
Agent's current location: Room B
Action taken: Moved to room A
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

--- Step 4 ---
Agent's current location: Room A
Action taken: Moved to room B
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

--- Step 5 ---
Agent's current location: Room B
Action taken: Moved to room A
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

Press any key to continue . . .
```

■ Both rooms are dirty

```
Is room A Dirty or Clean?
Dirty

Is room B Dirty or Clean?
Dirty

How many steps should the agent run? 5

--- Step 1 ---
Agent's current location: Room A
Action taken: Cleaned the room.
Current state of the environment: {'A': 'Clean', 'B': 'Dirty'}

--- Step 2 ---
Agent's current location: Room A
Action taken: Moved to room B
Current state of the environment: {'A': 'Clean', 'B': 'Dirty'}

--- Step 3 ---
Agent's current location: Room B
Action taken: Cleaned the room.
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

--- Step 4 ---
Agent's current location: Room B
Action taken: Moved to room A
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

--- Step 5 ---
Agent's current location: Room A
Action taken: Moved to room B
Current state of the environment: {'A': 'Clean', 'B': 'Clean'}

Press any key to continue . . .

Would you like to run the simulation again? (Type 'yes' to restart, or any other key to exit):
Exiting program. Goodbye!
```

❖ Optional Task

➤

➤ Code

```
# Optional Task
```

```
import random
```

```
print('\nProgrammed by: Arianne Beverly V. Marquez\n')
```

```
# TASK 1: THE ENVIRONMENT (Updated for Room C)
```

```
class VacuumEnvironment:
    def __init__(self):
```

```

        # Initialize three rooms using a Python
dictionary.
        self.rooms = {'A': 'Clean', 'B': 'Clean', 'C':
'Clean'}

    def set_initial_states(self):
        # Allow the user to set the initial state of
each room.
        # Adjusted input prompt to match the inline
format of the new screenshot.
        self.rooms['A'] = input("\nIs room A Dirty or
Clean? ").strip().capitalize()
        self.rooms['B'] = input("\nIs room B Dirty or
Clean? ").strip().capitalize()
        self.rooms['C'] = input("\nIs room C Dirty or
Clean? ").strip().capitalize()

    def is_dirty(self, room):
        # Function to check if a room is dirty.
        return self.rooms[room] == 'Dirty'

    def clean_room(self, room):
        # Function to clean a room.
        self.rooms[room] = 'Clean'
        print(f"Room {room} has been cleaned.")

# BONUS TASK: VISUALIZATION HELPER

def display_grid(rooms_dict, current_room):
    # Generates the [A-State] [B-State] [C-State] grid
output
    grid = []
    for room in ['A', 'B', 'C']:
        if room == current_room:
            grid.append(f"[{room}-Agent]")
        else:
            grid.append(f"[{room}-{rooms_dict[room]}]")

    print(" ".join(grid))

```

```

# TASK 2: THE RULE-BASED AGENT (Updated Logic)

class VacuumAgent:
    def __init__(self, environment):
        self.env = environment
        self.current_room = 'A'

    def perceive_and_act(self):
        # Rule 1: If the current room is dirty, clean
        it.
        if self.env.is_dirty(self.current_room):
            self.env.clean_room(self.current_room)

        # Rule 2: If the current room is clean, find
        dirty rooms.
        else:
            # Create a list of all rooms that are
            currently dirty
            dirty_rooms = [room for room, state in
self.env.rooms.items() if state == 'Dirty']

            if dirty_rooms:
                # Randomly pick a dirty room from the
                list to move to next
                self.current_room =
random.choice(dirty_rooms)
                print(f"Moving to room
{self.current_room}")
            else:
                # If the list is empty, all rooms are
                clean
                print("All rooms are clean. Agent is
                idle.")

# TASK 3: RUN THE SIMULATION

if __name__ == "__main__":
    while True:
        # Create a fresh environment and agent for
        each run
        env = VacuumEnvironment()
        agent = VacuumAgent(env)

```

```

        # Prompt user for initial room states
        env.set_initial_states()
        print()

        # Ask the user how many steps the simulation
        should run.
        try:
            steps = int(input("How many steps should
the agent run? "))
        except ValueError:
            print("Invalid input. Defaulting to 5
steps.")
            steps = 5

        # Run the agent in a loop for the specified
        steps.
        for step in range(1, steps + 1):
            print(f"\nStep {step}:")

            # Print the text-based grid before the
            agent takes action
            display_grid(env.rooms,
agent.current_room)

            # Agent perceives the room and acts
            agent.perceive_and_act()

            # Print the text-based grid after the
            agent takes action
            display_grid(env.rooms,
agent.current_room)

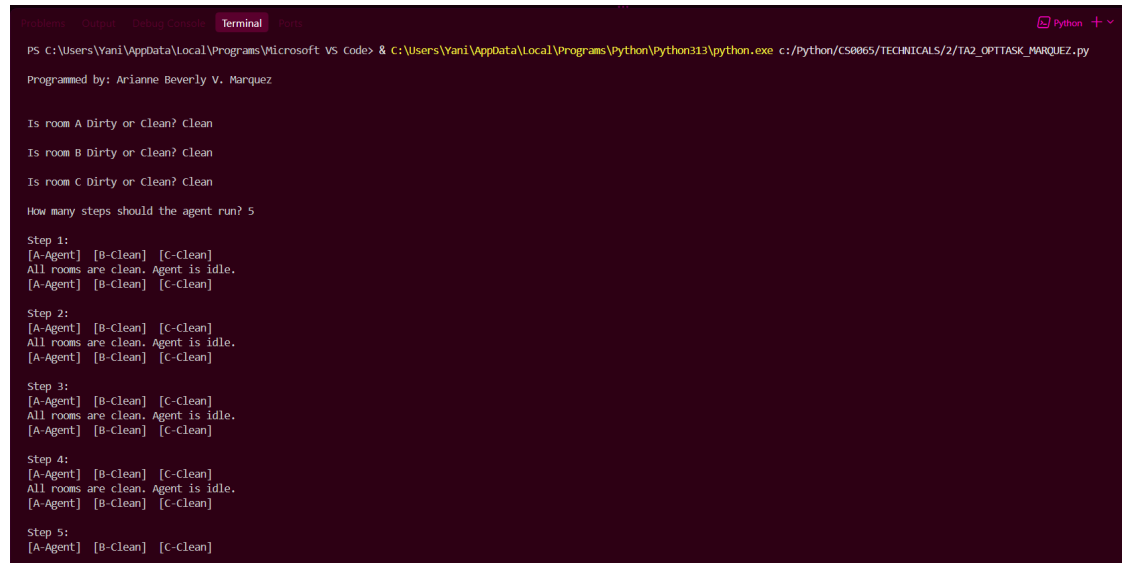
            print()
            input("Press any key to continue . . . ")

        # Ask the user if they want to run it again
        print()
        retry = input("Would you like to run the
simulation again? (Type 'yes' to restart, or any other
key to exit): ").strip().lower()

        if retry not in ['yes', 'y']:
            break

```

➤ Output



```
PS C:\Users\Vani\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Vani\AppData\Local\Programs\Python\Python313\python.exe c:/Python/CS0065/TECHNICALS/2/TA2_OPTTASK_MARQUEZ.py

Programmed by: Arianne Beverly V. Marquez

Is room A Dirty or Clean? Clean
Is room B Dirty or Clean? Clean
Is room C Dirty or Clean? Clean

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Clean] [C-Clean]
All rooms are clean. Agent is idle.
[A-Agent] [B-Clean] [C-Clean]

Step 2:
[A-Agent] [B-Clean] [C-Clean]
All rooms are clean. Agent is idle.
[A-Agent] [B-Clean] [C-Clean]

Step 3:
[A-Agent] [B-Clean] [C-Clean]
All rooms are clean. Agent is idle.
[A-Agent] [B-Clean] [C-Clean]

Step 4:
[A-Agent] [B-Clean] [C-Clean]
All rooms are clean. Agent is idle.
[A-Agent] [B-Clean] [C-Clean]

Step 5:
[A-Agent] [B-Clean] [C-Clean]
```

- All rooms clean

```
Problems Output Debug Console Terminal
Programmed by: Arianne Beverly V. Marquez

Is room A Dirty or Clean? Clean
Is room B Dirty or Clean? Clean
Is room C Dirty or Clean? Clean

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Clean] [C-Clean]
All rooms are clean. Agent is idle.
[A-Agent] [B-Clean] [C-Clean]

Step 2:
[A-Agent] [B-Clean] [C-Clean]
All rooms are clean. Agent is idle.
[A-Agent] [B-Clean] [C-Clean]

Step 3:
[A-Agent] [B-Clean] [C-Clean]
All rooms are clean. Agent is idle.
[A-Agent] [B-Clean] [C-Clean]

Step 4:
[A-Agent] [B-Clean] [C-Clean]
All rooms are clean. Agent is idle.
[A-Agent] [B-Clean] [C-Clean]

Step 5:
[A-Agent] [B-Clean] [C-Clean]
All rooms are clean. Agent is idle.
[A-Agent] [B-Clean] [C-Clean]
```

- One room is dirty, two are clean

```
Is room A Dirty or Clean? Dirty

Is room B Dirty or Clean? Clean

Is room C Dirty or Clean? Clean

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Clean] [C-Clean]
Room A has been cleaned.
[A-Agent] [B-Clean] [C-Clean]

Step 2:
[A-Agent] [B-Clean] [C-Clean]
All rooms are clean. Agent is idle.
[A-Agent] [B-Clean] [C-Clean]

Step 3:
[A-Agent] [B-Clean] [C-Clean]
All rooms are clean. Agent is idle.
[A-Agent] [B-Clean] [C-Clean]

Step 4:
[A-Agent] [B-Clean] [C-Clean]
All rooms are clean. Agent is idle.
[A-Agent] [B-Clean] [C-Clean]

Step 5:
[A-Agent] [B-Clean] [C-Clean]
All rooms are clean. Agent is idle.
[A-Agent] [B-Clean] [C-Clean]

Press any key to continue . . .
```

- Two rooms are dirty, one is clean

```
Problems Output Debug Console Terminal Ports

Is room A Dirty or Clean? Dirty

Is room B Dirty or Clean? Dirty

Is room C Dirty or Clean? Clean

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Dirty] [C-Clean]
Room A has been cleaned.
[A-Agent] [B-Dirty] [C-Clean]

Step 2:
[A-Agent] [B-Dirty] [C-Clean]
Moving to room B
[A-Clean] [B-Agent] [C-Clean]

Step 3:
[A-Clean] [B-Agent] [C-Clean]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Clean]

Step 4:
[A-Clean] [B-Agent] [C-Clean]
All rooms are clean. Agent is idle.
[A-Clean] [B-Agent] [C-Clean]

Step 5:
[A-Clean] [B-Agent] [C-Clean]
All rooms are clean. Agent is idle.
[A-Clean] [B-Agent] [C-Clean]

Press any key to continue . . .
```

- All rooms dirty

Is room A Dirty or Clean? Dirty

Is room B Dirty or Clean? Dirty

Is room C Dirty or Clean? Dirty

How many steps should the agent run? 5

Step 1:

[A-Agent] [B-Dirty] [C-Dirty]

Room A has been cleaned.

[A-Agent] [B-Dirty] [C-Dirty]

Step 2:

[A-Agent] [B-Dirty] [C-Dirty]

Moving to room B

[A-Clean] [B-Agent] [C-Dirty]

Step 3:

[A-Clean] [B-Agent] [C-Dirty]

Room B has been cleaned.

[A-Clean] [B-Agent] [C-Dirty]

Step 4:

[A-Clean] [B-Agent] [C-Dirty]

Moving to room C

[A-Clean] [B-Clean] [C-Agent]

Step 5:

[A-Clean] [B-Clean] [C-Agent]

Room C has been cleaned.

[A-Clean] [B-Clean] [C-Agent]

Press any key to continue . . .

```
Would you like to run the simulation again? (Type 'yes' to restart, or any other key to exit):  
Exiting program. Goodbye!  
PS C:\Users\Yani\AppData\Local\Programs\Microsoft VS Code>
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ↓

1. The rules that the agent follows are **"if the current room is dirty, clean it."** and **"if the current room is clean, move to another room."**
2. The agent makes decisions through the `perceive_and_act()` method to "look" at its current location/room. It queries the location/room using the `is_dirty()` function. **When the user enters "Dirty" to the prompt, the agent executes cleaning action. If the user enters "Clean", the agent skips cleaning and moves to another location/room.**
3. The agent just moves from one room to another to check if each room is clean. **If both rooms are already clean from the start, the agent just stays in the first room he/she is assigned to till the steps are done as the user confirms if the other rooms are clean or not.**
4. In the Optional (Bonus Task), what I did is I added another room (C). Here, I expanded the environment to hold additional keys. I also rewrote the agent's `move()` logic. This is because an `if/else` statement toggling between A and B no longer works for three or more rooms, **the agent must be programmed to scan the location/room's dictionary, compile a list of rooms currently marked "Dirty", and either move to the next one in sequence or select one randomly.**